Capstan RL Field Guide

How the v4 quadruped learns to move, read from the code

CapstanPedal/fullbuild_v4/rl2 · 26 October 2026 · code links point at commit 231ca53, FastSAC links at e0ecfe6

The whole thing in one paragraph. A URDF made from the CAD becomes a MuJoCo simulation. 8,192 copies of the robot run at once on the GPU. Every 20 ms each robot's *policy* (a neural network) reads 153 noisy sensor numbers and outputs 12 joint targets, plus random noise so it tries something slightly different each time. Every step is scored by a weighted sum of reward terms. A second network, the *critic*, predicts how much reward is coming. After 24 steps of all 8,192 robots, PPO makes the actions that did *better than the critic expected* more likely and the rest less likely. Repeat 1,500 times (about 15–30 minutes on the RTX 4090) and the noise becomes a walk. Only the policy runs on the robot. **Newer and better on terrain:** FastSAC (§15) keeps every step in a replay buffer, learns *how good each action is* rather than how good each state is, and reuses each sample ~16 times: 88 % held-out on terrain against PPO's 71 %.

Contents

1 The map **2**

2 From a URDF to 8,192 robots on a GPU **3**

3 One step of one robot **3**

3.1 What the policy sees: 3 frames of 51 numbers . . . 3

3.2 Act, with noise . . . 4

3.3 Action → joint targets → torque . . . 4

3.4 Score it . . . 4

4 The core ideas, before any algorithm **4**

5 Did that noisy try get reward? The critic, the advantage, PPO **5**

5.1 Return and value . . . 5

5.2 Advantage: better or worse than expected? . . . 6

5.3 GAE: estimating the advantage from 24 steps . . . 6

5.4 The PPO update . . . 6

6 What happens next: four loops **6**

6.1 Choosing the next scenario . . . 7

7 Logs and saving **7**

8 Base RL: the reward you actually care about **7**

9 Reward shaping: terms that point the way **7**

9.1 Why a bell curve . . . 8

9.2 Shaping in the other tasks . . . 8

10 Reward tuning: picking the numbers **8**

11 An RL controller vs the model-based one **9**

12 Continuous commands **9**

13 How does it know trot vs walk vs crawl? **9**

14 Privileged learning: a teacher that sees, a student that feels **10**

15 FastSAC: off-policy RL on thousands of robots **11**

15.1 Idea 1: learn Q and keep a replay buffer . . . 11
15.2 Idea 2: the actor follows the critic's slope . . . 11
15.3 Idea 3: maximum entropy (the "soft" in SAC) . . . 11
15.4 Idea 4: two critics, and how to combine them . . . 12
15.5 Idea 5: a distribution, not a number (C51) . . . 12
15.6 Idea 6: n-step returns . . . 12
15.7 Idea 7: scale, and the settings that made it work . . . 12
15.8 What happened (Experiment 6, queue6–7, 26 Oct 2026) . . . 12

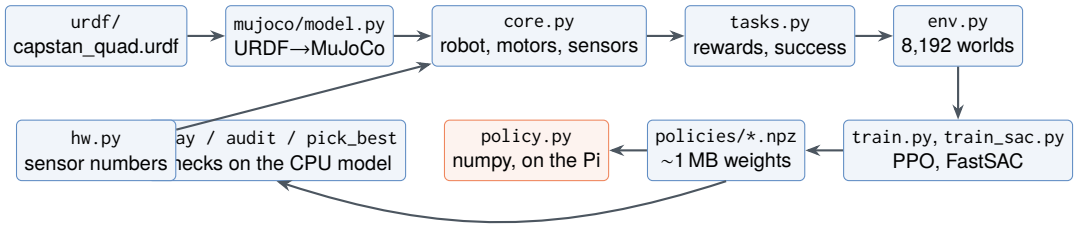
16 Is it good? How a policy is judged **13**

17 A new build version **13**

18 Words **14**

19 Check yourself **14**

1 The map



Everything for the learned controller lives in `rl2/`. It borrows the robot from two folders next to it: `urdf/` (the description generated from CAD) and `mujoco/` (which turns the URDF into a simulation and also holds the hand-written controller `gait.py`). `rl/` is the first version (walking only, ideal sensors); read `rl2`.

File	Its job
`hw.py`	The sensors as numbers: 50 Hz policy, 4–20 ms delay, gyro noise, encoder resolution, optical-flow dropouts, battery sag. Replace the assumed values with measurements.
`core.py`	Builds the model twice (light "physics" for the GPU, "full" with every mesh for checking); PD loop and motor model; `sense()`, which turns true simulator state into what the sensors would report; terrain height.
`tasks.py`	The five tasks (walk, terrain, rear, handstand, balance): what an episode is, the scenario bank, every reward term, success and fall.
`env.py`	Steps a task: `GpuEnv` runs thousands of worlds on the GPU, `CpuEnv` one world in the full model.
`train.py`	PPO: actor and critic, rollout, update, logging, held-out evaluation, snapshots. `--teacher=1`: a teacher (§14).
`train_sac.py`	FastSAC: off-policy, replay buffer, twin distributional Q critics, entropy-tuned (§15). Same tasks and outputs.
`distill.py`	Teacher → blind student by imitation (DAgger). No reward.
`policy.py`	The trained actor in numpy. This is what would run on the Raspberry Pi.
`play.py, audit.py, pick_best.py`	Held-out success on the full model, stress tests, choosing the best snapshot.
`terms.py, progress.py, showreel.py, compare_film.py`	Looking at runs: which reward terms dominated, films of snapshots, the all-skills film, teacher/student/blind side by side.
`run_queue.sh, experiments/`	Train a batch on cake; every batch we ran is a list file.
`check_robot.py`	Does a build's robot fit rl2? (§17)
`wandb_sync.py`	Streams the CSV logs to Weights & Biases; training itself needs no account.

2 From a URDF to 8,192 robots on a GPU

1. **Read the URDF as XML.** `mujoco/model.py` [L33-L46] parses `capstan_quad.urdf` and pulls out the `<mimic>` tags: the motor rotors spin at −9× their output, and MuJoCo cannot read mimic, so they come back later as joint equalities.

```
root = ET.parse(urdf).getroot()
for j in root.findall("joint"):
    m = j.find("mimic")
    ...
spec = mujoco.MjSpec.from_string(ET.tostring(root, encoding="unicode"))
```

2. **Add what a URDF cannot say** (`mujoco/model.py` [L48-L125]): a free joint so the torso can fly (URDF roots are fixed), a floor and lights, 12 actuators (motor C on ab/ad, B on the hip, A on the crank), the four-bar loop closed with a *connect* equality, reflected rotor inertia (*armature*) on the driven joints, and foot sites.
3. **Make the learning version.** `core.build_spec("physics")` (`rl2/core.py` [L60-L98]) deletes every visual mesh and the rotor joints: the GPU only needs collision shapes, and the rotor inertia is already in the armature. The "full" version keeps everything and adds the flow and range sensor boards. Tasks add *extras*: 64 mocap boxes for the terrain course, or a ball and board.
4. **Find things by name.** `Layout` (`rl2/core.py` [L115-L146]) records where the 12 driven joints, four feet and the torso are in the compiled model, and computes the standing pose from the leg IK in `gait.py`. This is why a new build works as long as the names stay.
5. **Copy it onto the GPU** (`rl2/env.py` [L26-L42]). `GpuEnv` compiles the model and hands it to MuJoCo Warp through MJX; one `Data` per robot, all stepped together with `jax.vmap`. Friction, mass and the terrain differ per robot, so they are passed per world.

3 One step of one robot

A *step* is one decision, 50 times per simulated second. Between decisions physics runs 10 times (500 Hz) with the joint PD loop holding the targets.

3.1 What the policy sees: 3 frames of 51 numbers

Every number passes through `sense()` (`rl2/core.py` [L216-L241]), so it is noisy, biased, late, quantised or missing, as on the robot.

Group	n	Source on the robot, and the sim's errors
Rotation rate (gyro) ×0.25	3	pi3hat IMU; 0.02 rad/s noise, ±0.02 rad/s bias per episode
Gravity direction	3	pi3hat attitude filter; ±1.5° tilt error per episode, 0.4° noise
Ground speed (fwd, side) ×2	2	PMW3901 optical flow × range: one step old, ±8 % scale, dropouts held, 15 % of episodes dead
Flow valid	1	the flow sensor's quality flag
Height $(h - h_0) \times 5$	1	VL53L1X time-of-flight range; 5 mm + 1 % noise
Command	3	joystick: forward, sideways (m/s), turn (rad/s); scaled ×2, ×2, ×0.25
Joint angles − standing pose	12	moteus encoders: 14-bit through 9:1, ±0.01 rad calibration offset
Joint speeds ×0.05	12	moteus velocity estimate, 0.05 rad/s noise
Last action	12	the policy's own previous output
Trot clock	2	sin, cos of a 0.3 s cycle (walking tasks; zeros otherwise)

Three frames are stacked (153 numbers), so the policy can see short trends. Each input is then normalised by a running mean and variance and clipped to ±10.

3.2 Act, with noise

The actor is an MLP, $153 \to 512 \to 256 \to 128 \to 12$ with ELU activations. It outputs a mean $\mu(o)$; during training a Gaussian sample is taken with a learned, state-independent standard deviation $\sigma = e^{\texttt{log_std}}$ (initially $e^{-1} \approx 0.37$):

$$a \sim \mathcal{N}\big(\mu_\theta(o),\ \mathrm{diag}\,\sigma^2\big), \qquad \log \pi_\theta(a \mid o) = \sum_{j=1}^{12} \Big[-\tfrac{1}{2}\Big(\tfrac{a_j - \mu_j}{\sigma_j}\Big)^2 - \log \sigma_j - \tfrac{1}{2}\log 2\pi \Big].$$

```
mu = actor_mean(p, nm, obs)
std = jp.exp(p["log_std"])
a = mu + std * jax.random.normal(k_a, mu.shape)     # each robot gets its own noise: exploration
```

On the robot (and in every evaluation) the noise is off: the action is $\mu(o)$.

3.3 Action → joint targets → torque

Each action moves a joint target from the standing pose: 0.2 rad per unit on ab/ad, 0.6 on hip and crank, clamped to the firmware limits (`rl2/core.py` L187-L193). For 10 sub-steps the PD loop and motor model (`rl2/env.py` L51-L70) compute

$$\tau = k_p(q^* - q) - k_d\,\dot{q}, \qquad |\tau| \le \tau_{\max}\, s\, \mathrm{clip}\Big(2\big(1 - \tfrac{|\dot{q}|}{\omega_0 b}\big), 0, 1\Big),$$

with $k_p = 80/60/60$, $k_d = 1.5/2/2$ per joint, s the motor strength (0.75–1.05 per episode), b the battery sag (0.83–1.0), and the crank output A limited further by how hard the coupler can push. Full torque up to half the no-load speed, falling to zero at it. The new targets only apply after the episode's control delay (4–20 ms); before that the old ones hold.

```
tgt = jp.where(i < delay, aux["prev_target"], target)
tau = C.torque(jp, pos, vel, tgt, aux["kp"], aux["kd"], aux["strength"], aux["w0"], L.isA)
d = mjx.step(mx, d.replace(ctrl=d.ctrl.at[L.act].set(tau)))
```

3.4 Score it

The task's `end()` reads the *true* state (the simulator knows everything; the policy does not), computes every reward term, decides fallen / success / time-out, and senses the next observation (`rl2/tasks.py` L166-L194 for walk, `rl2/tasks.py` L118-L133 for the shared `finish()`).

4 The core ideas, before any algorithm

Every method in this guide (PPO, FastSAC, the teacher–student pair) is built from the same five pieces. Learn these and each algorithm becomes a choice of which pieces to estimate and how. The standard textbook is Sutton and Barto [1]; OpenAI's *Spinning Up* [2] is the shortest good tour.

1. The problem (an MDP). At each step the world is in a state s_t, the robot sees an observation o_t (part of s_t, with noise), its policy picks an action $a_t \sim \pi(\cdot \mid o_t)$, the world moves to s_{t+1} and pays a reward r_t. Nobody tells the robot the right action; it only ever sees rewards.

2. The goal: the return. Maximise $J(\pi) = \mathbb{E}_\pi\big[G_0\big]$ with $G_t = \sum_{l \ge 0} \gamma^l r_{t+l}$. The discount $\gamma < 1$ keeps the sum finite and sets the horizon: about $1/(1-\gamma)$ steps, so 100 steps (2 s) at $\gamma = 0.99$ and 33 steps (0.7 s) at $\gamma = 0.97$.

3. Two ways to score a situation.

$$V^\pi(s) = \mathbb{E}_\pi\big[G_t \mid s_t = s\big] \qquad Q^\pi(s,a) = \mathbb{E}_\pi\big[G_t \mid s_t = s,\ a_t = a\big] \qquad A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s).$$

V: how good is it to *be* here. Q: how good is it to *do this* here, then carry on with π. The advantage A: how much better this action is than the policy's usual. PPO learns V; FastSAC learns Q.

4. The Bellman equation: values are consistent with themselves. The return is one reward plus the (discounted) return from the next state, so

$$Q^\pi(s,a) = \mathbb{E}\big[\, r + \gamma\, Q^\pi(s',a')\,\big], \qquad a' \sim \pi(\cdot \mid s').$$

This turns learning a value into regression on its own predictions: *temporal-difference* (TD) learning. Take a stored transition, compute the *target* $y = r + \gamma Q(s',a')$ with the current network, and move $Q(s,a)$ toward y. For example, with $r = 0.12$, $\gamma = 0.97$ and $Q(s',a') = 4.0$, the target is $0.12 + 0.97 \times 4.0 = 4.00$. If the critic said $Q(s,a) = 3.6$, the TD error is +0.4: this step went better than predicted, so raise $Q(s,a)$. The catch is that the target moves when Q moves (chasing your own tail), which is why every Q-learning method keeps a slowly updated *target network* for y [7].

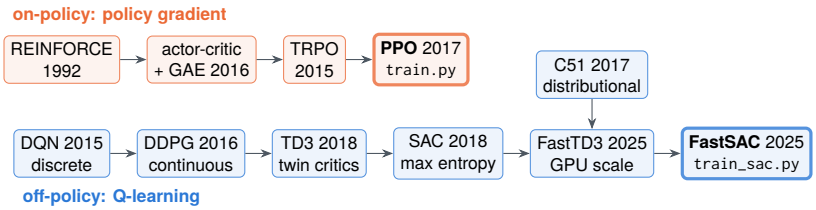
5. The policy gradient: push up what beat expectations. For a stochastic policy [3, 5],

$$\nabla_\theta J = \mathbb{E}\big[\nabla_\theta \log \pi_\theta(a_t \mid o_t)\ A(s_t,a_t)\big].$$

Read it as: for every action tried, make it more likely in proportion to how much better than expected it turned out. No model of the physics, no derivative of the reward; only the log-probability of the action you took. PPO is this, made safe for big steps (§5).

The split that matters: on-policy vs off-policy.

	On-policy (PPO)	**Off-policy** (DQN, DDPG, TD3, SAC, FastSAC)
Learns from	only data from the *current* policy	any past data, kept in a *replay buffer*
Each sample is used	a few times (5 passes here), then thrown away	many times (about 16 times here)
The critic learns	$V(s)$, from real returns (GAE)	$Q(s,a)$, from its own next-step predictions (Bellman)
The policy improves by	the policy gradient with advantages	following $\partial Q/\partial a$: pick actions the critic rates higher
Strength	simple, robust, one main knob	data-efficient; can keep improving from old experience
Weakness	needs fresh data every update	bootstrapping can go unstable: errors feed on themselves



5 Did that noisy try get reward? The critic, the advantage, PPO

PPO (Schulman et al. [6]) is the policy gradient of §4 with advantages from GAE [5] and a clip that stands in for TRPO's trust region [4]. It is what legged robots have mostly been trained with since Rudin et al. [21] ran thousands of them on one GPU.

Reward arrives *every step*; there is no separate check at the end. `finish()` multiplies each raw term by its weight and adds them up, then scales by 0.1:

$$r_t = 0.1 \sum_k w_k\, x_{k,t}.$$

Success and falls are terms in this sum (+10, −10), not a separate signal: PPO never looks at "success" on its own.

5.1 Return and value

What we want to maximise is the *discounted return*, the reward from here on with later rewards counted less:

$$G_t = \sum_{l \ge 0} \gamma^l r_{t+l}, \qquad \gamma = 0.99 \ \ \text{(a reward 100 steps = 2 s ahead counts } 0.99^{100} \approx 0.37\text{)}.$$

The *critic* $V_\phi(s)$ learns to predict G_t. It may read things the robot cannot (the 22 *privileged* numbers: true velocity, true height, foot contacts, friction, added mass, motor strength, gains, delay, flow scale error, and per task 6 more; terrain now reads a 30-point height scan and foot clearances instead). The critic only exists in training, so this is allowed.

5.2 Advantage: better or worse than expected?

A balanced, standing robot always collects decent reward; that says nothing about whether its last action was good. So PPO asks whether the outcome beat the critic's prediction:

$$A_t \approx G_t - V(s_t).$$

Robot	Its noise pushed it…	Expected	Got	Advantage	PPO will…
#17	front-left foot down a bit earlier	0.80	1.00	+0.20	make that action more likely
#4,203	hip harder, body pitched	0.80	0.50	−0.30	make it less likely
#6,050	about the mean	0.80	0.81	+0.01	barely change

5.3 GAE: estimating the advantage from 24 steps

After a 24-step rollout we do not know G_t (episodes are 400 steps). *Generalised advantage estimation* mixes real rewards with the critic's guess of the rest, walking backwards (`rl2/train.py` L304-L311):

$$\delta_t = r_t + \gamma\,(1-d_t)\,V(s_{t+1}) - V(s_t), \qquad \hat{A}_t = \delta_t + \gamma\lambda\,(1-d_t)\,\hat{A}_{t+1}, \qquad \lambda = 0.95,$$

where $d_t = 1$ if the episode ended at t (no future after a fall). The critic's training target is $\hat{R}_t = \hat{A}_t + V(s_t)$. One subtlety: a *time-out* is not a real ending (the robot would have kept going), so on a time-out the code adds $\gamma V(s_{t+1})$ to the reward: it bootstraps from the last state instead of treating the future as zero.

```
def gae(nxt, t):
    adv_n, v_n = nxt
    nd = 1.0 - t["done"]                              # no future after a fall
    delta = t["r"] + o["gamma"] * v_n * nd - t["v"]   # reward + expected next - expected now
    adv = delta + o["gamma"] * o["lam"] * nd * adv_n
    return (adv, t["v"]), adv
```

5.4 The PPO update

For each stored action, the *ratio* $\rho_t(\theta) = \pi_\theta(a_t \mid o_t)/\pi_{\text{old}}(a_t \mid o_t)$ says how much more likely the new network makes it than the one that chose it. With advantages normalised per minibatch ($\hat{A} \leftarrow (\hat{A} - \bar{A})/\mathrm{sd}(\hat{A})$), PPO minimises (`rl2/train.py` L283-L294)

$$L(\theta,\phi) = -\,\mathbb{E}_t\Big[\min\big(\rho_t\hat{A}_t,\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big] + c_v\,\mathbb{E}_t\big[(V_\phi(s_t) - \hat{R}_t)^2\big] - c_e\,\mathcal{H}[\pi_\theta],$$

with $\epsilon = 0.2$, $c_v = 1$, $c_e = 0.004$. The clip stops one batch from moving the policy more than ±20 % on any action; the entropy bonus $\mathcal{H} = \sum_j(\log\sigma_j + \frac{1}{2}\log 2\pi e)$ keeps some exploration. Each iteration makes 5 passes over the batch in 4 minibatches of 49,152, with Adam and gradient-norm clipping at 1.

Adaptive learning rate (`rl2/train.py` L365-L368): the approximate KL divergence $\mathbb{E}[(\rho - 1) - \log\rho]$ measures how far one update moved the policy. Above 2×0.01 the learning rate is divided by 1.5; below $0.01/2$ it is multiplied by 1.5 (bounds 10^{-5}–10^{-2}).

> **One try teaches almost nothing.** The policy moves because 196,608 noisy actions per update average out toward the better ones. Any single robot's experience is mostly noise.

6 What happens next: four loops

Word	How long	What it is here
Step	20 ms	One action, 10 physics steps, one reward.
Episode	≤400 steps	One try at one scenario: walk 8 s, poses 6 s. Ends early on a fall, or (terrain) on reaching the goal.
Iteration	24 steps	Every robot takes 24 steps (8,192 × 24 = 196,608 samples), then one PPO update.
Run	~1,500–2,000 iterations	One training job: 300–400 M steps, 15–30 minutes on the 4090 at 200–235 k steps/s.

The weights are fixed during the 24-step rollout and change only at the update. Episodes do *not* restart at an update: a robot halfway through a walk keeps walking with the slightly better policy. When a robot's episode ends it is reset in the same step into a new scenario. All robots run in lock-step on the GPU, so the reset is a *masked swap*: where `done` is true take the fresh state, else keep the old (`GpuEnv.respawn`).

```
new_cfg = jax.random.randint(k_c, (n,), 0, n_cfg)          # uniform; or SGS / curriculum
d2, aux2, mp, obs_n, priv_n = env.respawn(bank, d2, aux2, mp, obs2, priv2, done, new_cfg, k_r)
# inside: sel = lambda new, old: jp.where(done[:, None...], new, old)
```

A *scenario* is one row of the *bank*: everything random about an episode, fixed in advance. Commands, floor friction (0.4–1.25), added mass (−0.5 to +2 kg), motor strength, gain errors, battery sag, a shove (time and velocity), and for terrain the course itself. 1,040 scenarios for training; 4,160 different ones (another seed) for the held-out test.

6.1 Choosing the next scenario

- **Uniform**: any scenario, equally likely. What every deployable policy used.
- **SGS** (success-guided sampling, Zhang et al. [27]): each scenario keeps its last 100 outcomes; with p_i their mean,

$$w_i = \max\left((p_i+\varepsilon)^{kt}(1-p_i+\varepsilon)^{k(1-t)},\ \varepsilon\right), \quad P(i) \propto \exp\left(\log(w_i+\varepsilon)/T\right), \quad t = 0.66,\ k = 5,\ T = 2,$$

  which peaks at scenarios succeeded about two times in three: hard but possible.
- **Level curriculum** (Rudin et al. [21]): each robot keeps a level per terrain type, up one on success, down one on failure.

What we found: over 14 runs SGS never beat uniform and lost 15–20 points on terrain (a scenario that fails early is sampled less, so it is never learned: the bank *freezes*). The curriculum gave +3 points, within seed noise; robots settled at level ∼1, because up-on-success/down-on-failure settles where success is 50 %.

7 Logs and saving

When	File (on cake)	What is in it
Start	`logs/<name>_opts.json`	Every option and the reward weights used: a run can always be reproduced.
Every iteration	`logs/<name>.csv`	Steps, speed, learning rate, mean reward, success and fall rate of finished episodes, KL, critic loss, noise size, every reward term as `r.<term>`.
Every 10 it.	`logs/<name>.out`	A printed line with the four biggest reward terms.
Every 50 it.	`logs/<name>_eval.jsonl`	Held-out test: 4,160 unseen scenarios, mean action, success per terrain and level.
Every 50 it.	`policies/snaps/<name>/<M>.npz`	An actor snapshot (teacher runs also keep a `.pkl`).
Every 50 it., end	`policies/<name>.npz, .pkl`	`.npz`: actor + input normalisation, ∼1 MB, deployable. `.pkl`: everything, to resume.
After the run	`logs/<name>.post, .audit, .pick`	`run_queue.sh` films it, evaluates it on the full CPU model, stress-tests it, picks the best snapshot as `<name>_best.npz`.

Only the deployable `_best.npz` files live in git; logs, checkpoints and films stay on cake. `wandb_sync.py --follow` streams the CSVs to Weights & Biases while a run trains.

8 Base RL: the reward you actually care about

The plainest reward is the goal itself: +10 for success, −10 for a fall, nothing otherwise. That is *sparse* (base) reward, and it is in every task as the `success` and `fall` terms. On its own it almost never works for a legged robot: a random policy falls within a second, every time, never stumbles into an 8 s walk that tracks the command, so success is never seen and the only lesson is "falling is bad". The policy learns to crouch and sit still.

9 Reward shaping: terms that point the way

Shaping adds *dense* terms that give partial credit every step, so a slightly better policy gets slightly more reward long before it ever succeeds. The whole walk reward (`rl2/tasks.py` L141-L142 weights, `rl2/tasks.py` L170-L181 raw values, `rl2/tasks.py` L111-L116 shared terms):

Term	Raw value each step	Weight	Why
`track_lin`	$\exp(-\|v_{xy} - c_{xy}\|^2/0.25)$	+1.5	The goal as partial credit
`track_yaw`	$\exp(-(\omega_z - c_\omega)^2/0.25)$	+0.75	Turning on command
`gait`	share of feet whose contact matches the trot clock	+0.5	Asks for a trot (§13)
`air`	at touch-down: air time − 0.15 s	+1.0	Real steps, not shuffling
`bounce`	v_z^2	−2.0	Smooth body
`angvel`	$\omega_x^2 + \omega_y^2$	−0.05	No rocking
`tilt`	$g_x^2 + g_y^2$ (gravity off vertical)	−5.0	Body level
`height`	$(z - z_{\text{stand}})^2$	−20	No crouch-walking
`torque`	$\sum \tau^2$	-2×10^{-4}	Energy, motor heat
`action_rate`	$\sum(a_t - a_{t-1})^2$	−0.01	No jitter: gears would suffer
`slip`	$\sum$ speed^2 of feet in contact	−0.2	Planted feet
`idle_pose`	distance from standing pose, when the command is zero	−0.2	Stand still properly
`fall`	1 on the step it falls	−10	Base reward
`success`	1 at the end of a successful episode	+10	Base reward

9.1 Why a bell curve

Tracking terms use $\exp(-e^2/\sigma)$ rather than $-e^2$: full credit at the target, smoothly less further away, never a cliff, and bounded, so one term cannot drown the rest. With $\sigma = 0.25$: error 0 → 1.00; 0.25 m/s → 0.78; 0.5 m/s → 0.37; 1 m/s → 0.02. At 0.25 m/s off, track_lin pays $0.78 \times 1.5 \times 0.1 = 0.117$ per step, about 47 over an 8 s episode, against the ±1 of success/fall after scaling: the dense terms carry most of the signal.

9.2 Shaping in the other tasks

Rear, handstand: `pose` pays for the nose pointing up (down) as a bell curve around the target angle; `feet` pays for the right pair being off the ground; `roll`, `drift` are penalties (`rl2/tasks.py` L343-L344). **Balance**: `alive` pays 1 per step not fallen, `board_level` penalises a tilted board, `feet_on` pays for feet on the board (`rl2/tasks.py` L417-L418). **Terrain**: the walk terms with gentler tilt and height penalties, so stairs and slopes are not punished (`rl2/tasks.py` L279-L280).

The policy optimises what you wrote, not what you meant. The first handstand held the body at 45–50°, right at the 50° success bar, because the pose bell curve paid nearly as much there as at 64°. "Successful" 14 % of the time. That is a shaping bug.

10 Reward tuning: picking the numbers

Shaping picks the terms; tuning picks their weights (and widths such as the 0.25). The loop:

1. Train with the current weights.
2. Watch the film and read the term report (`python terms.py walk walk_try`): which term dominates, which never moved.
3. Find the term that pays for the wrong thing or does not punish it enough. Change one or two weights.
4. Retrain without editing code: `python train.py --task=walk --w=slip=-0.5,air=2.0 --name=walk_try`. An unknown term name is an error (`rl2/tasks.py` L50-L55), so typos cannot silently do nothing.

What we saw	What changed	Result
Handstand hovering at the 50° bar	pose target 0.90 → 0.87 (≈ 60°), width 0.15 → 0.05, new `margin` term (+1.0) rising smoothly from 50° to 58°	best snapshot 35 % → 82 %
Handstand resting on the body's front edge	a body corner touching the ground counts as a fall	no more leaning
Torque constraint 85 % (`--cat=0.25`)	episodes cut when a motor passed 85 %	0 %: the working handstand needs more than 85 % on 41 % of steps. Too strict a number kills the skill

Tuning is not the only lever. Walk fell 3 times in 4 when the flow sensor died. That was fixed by training with it dead in 15 % of episodes (`hw.FLOW_DEAD`): falls 75 % → 0 %. That is *domain randomisation*: changing what the robot practises rather than what it is paid for.

11 An RL controller vs the model-based one

Both send joint targets to the same PD loop and motor model; they differ in how they choose them.

`mujoco/gait.py`: written by a person.

- Picks a gait from a table: which feet are down when.
- Plans each foot: stance feet slide back at the commanded speed; swing feet land where the Raibert rule says (half a stride ahead, corrected for speed error), lifted on an arc.
- Leg inverse kinematics turns foot positions into joint angles.
- Feed-forward force $J^\top F$ so each stance leg carries its share of the weight, plus roll and pitch correction.
- Uses the true body state from the simulator.
- Predictable and explainable; does only what its author thought of. No handstand, no board on a ball; an unplanned shove or step can tip it.

`rl2`: found by trial and error.

- One network: 153 sensor numbers in, 12 joint targets out, 50 times a second.
- No foot planning, no IK, no gait table: the stepping pattern is whatever the reward favoured.
- Sees only the noisy, late readings the real robot will have.
- Trained on thousands of variations (friction, mass, weak motors, shoves, delay), so it copes with them.
- Does skills that are hard to write by hand (handstand, balance board, blind stairs). Cannot explain itself, can find loopholes in the reward, and is only as good as the simulation is honest.

12 Continuous commands

Both controllers take the same three joystick numbers: forward and sideways speed (m/s) and turn rate (rad/s). They are continuous: any mix, any value in range. A diagonal walk is two at once, e.g. forward 0.4, sideways 0.3.

In `gait.py` the command goes straight into the foot plan: stance feet slide backwards along the command vector (diagonally, for a diagonal) and the Raibert rule places swing feet ahead along it. The diagonal works because the geometry says so.

In the RL policy the command is 3 of the 51 numbers in each frame. The policy has no idea what they mean; it learns that `track_lin` pays when its body speed equals them. It can walk diagonally because it was trained on diagonals: forward $-0.5 \ldots 1.2$, sideways $-0.4 \ldots 0.4$, turn $-1.2 \ldots 1.2$, 30 % along one axis only, 12 % standing still, switching to new values 4 s into each episode (`rl2/tasks.py` L143-L158). Consequence: the policy is only good inside the ranges it saw. Ask for 2 m/s and it will do *something*, but nobody rewarded it. Ranges are a training choice, not physics.

In **terrain** a simulated operator gives the commands: speed ramps to the target, sideways and turn commands steer back to the course's centre line (`rl2/tasks.py` L328). In `teleop.py` the keyboard sets them.

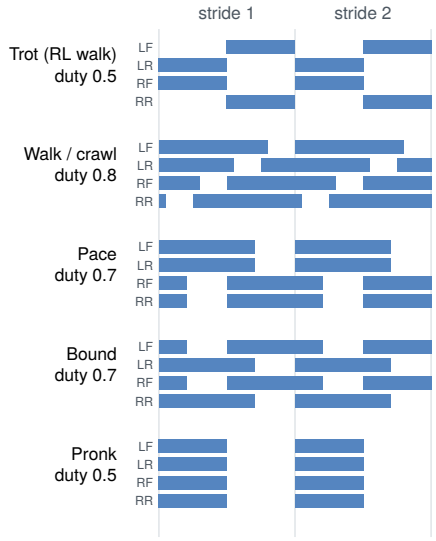
13 How does it know trot vs walk vs crawl?

It does not, unless the reward says so. A gait is a timing pattern: which feet are down at each moment of the stride. Given only "track the speed, do not fall", RL finds whatever pattern pays most: maybe a trot, maybe a bound, a pronk, or a shuffle the simulator allows and a real robot should not do. The walk task asks for a trot two ways:

1. **A clock in the input**: the last 2 numbers of each frame are sin, cos of a 0.3 s cycle, so the policy knows where in the stride it is.
2. **A reward for following it**: `gait` pays the share of feet whose contact matches the clock. Pod i has phase offset o_i; it should be down while $(\phi + o_i) \bmod 1 < d$, with duty $d = 0.5$.

```
OFF = np.array([0.5, 0.0, 0.0, 0.5])     # per pod: LF, LR, RF, RR -> diagonal pairs: a trot
want = (phase + C.OFF) % 1.0 < 0.5       # True = this foot should be on the ground
gait = jp.mean(contact == want)
```

(The robot walks toward $-X$, so the "F" pods are its hind legs.) Other gaits are other offsets and duties; these are the ones `gait.py` uses (bars: foot on the ground, two strides):

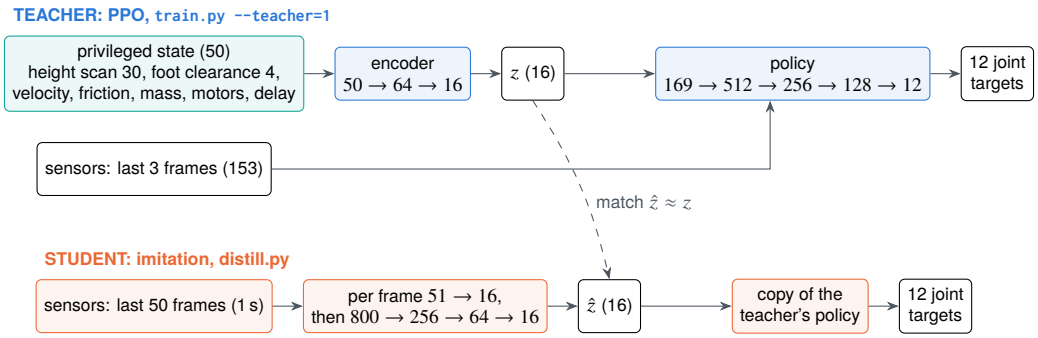


Three ways to get gaits from RL, from most control to least:

1. **Fix one gait** with a clock and a gait reward, as here. For a crawl: offsets $(0, 0.25, 0.5, 0.75)$ and duty 0.75 so three feet are always down (today the 0.5 duty is written into the `want` line, so it would become a parameter).
2. **Make the gait a command**: put offsets and duty in the observation, draw a different gait per episode; one policy learns them all and the operator picks at run time ("Walk These Ways" [25]).
3. **Let it emerge**: drop the gait terms, keep an energy penalty. Fu et al. [26] found energy alone gives a walk at low speed, a trot at medium and a gallop-like gait at high, as animals do. Less control over what you get.

14 Privileged learning: a teacher that sees, a student that feels

The idea (Lee et al. [22]; Kumar et al., RMA [23]). Learning to walk blind over terrain is hard partly because the policy has to work out the ground from its legs *and* learn to act on it, at once, from a noisy reward. Split the two:



Teacher. Ordinary PPO, but its policy also reads the privileged state through a small encoder to a 16-number latent z. For terrain that state is a height scan of the course (10 points from 0.3 m behind to 1.05 m ahead × 3 across, relative to the body's height), each foot's height above the ground under it, and the hidden physics (true velocity, friction, added mass, motor strength, gains, delay). The robot has no sensor for any of it, so the teacher is not deployable: `policy.py` refuses it.

Student. Blind. It reads the last 50 frames (1 s) of the robot's real sensors. A shared per-frame layer ($51 \to 16$) compresses each frame; an MLP over the 50 compressed frames estimates $\hat{z}$; the policy is initialised as a *copy* of the teacher's and reads [last 3 frames, $\hat{z}$]. Trained by imitation, with no reward at all:

$$\mathcal{L} = \mathbb{E}_{s\sim d^{\pi_\beta}}\Big[\|\pi_S(h_s) - \mu_T(o_s, p_s)\|^2 + \|\hat{z}(h_s) - z(p_s)\|^2\Big], \qquad \pi_\beta = \begin{cases} \mu_T & \text{with prob. } \beta \\ \pi_S & \text{otherwise} \end{cases},$$

where h is the 1 s history, p the privileged state, and β falls from 1 to 0 over the first 20 % of training. This is **DAgger** [24]: the states come from the *student's own* driving, so it learns what to do after its own mistakes, which plain copying of teacher trajectories never shows it.

What happened (Experiment 5, `experiments/queue5.list`, 26 Oct 2026; all held-out):

	Peak, GPU (4,160)	Best snap., CPU (200)	Last snap., CPU (200)	Stress audit (240)	Gaps (GPU)
Teacher (sees the scan)	**80.9 %**	not deployable	–	–	**52 %**
Student (blind, distilled)	78.3 %	77.0 %	**74.5 %**	**76 %**	12 %
Blind PPO (control)	70.7 %	77.0 %	55.5 %	59 %	11 %

- **Seeing is worth about 10 points, nearly all on gaps and mixed courses.** A blind robot cannot know a gap is there until a foot finds nothing; no imitation can fix that. In the films the teacher lengthens its stride before the gap; student and blind both stop at the edge.
- **The student keeps what can be felt**: stairs up 86 % vs 74 % blind; rough, slope, obstacles slightly better.
- **Best against best, blind success did not move** (77.0 % both, ±3 points at 200 episodes).
- **Distillation is stable where PPO is not.** The teacher slumped 81 → 45 %, the blind run 71 → 55 %; the student stayed in 63–78 % throughout. Supervised learning has a fixed target, which points the late-slump problem at the RL loop itself.
- **Next**: give the robot part of the scan. A forward-facing 8×8-zone range sensor (VL53L5CX) would let a student see gaps coming.

15 FastSAC: off-policy RL on thousands of robots

Why try it. Every PPO run on terrain peaked around 100–130 M steps and then lost 15–30 points (§14). The teacher–student experiment showed a supervised learner with a fixed target does not slump, which pointed at the RL loop. FastTD3 and FastSAC (Seo, Abbeel et al. [19, 20], Berkeley) showed that off-policy RL, long thought too slow and fragile for massively parallel simulation, can beat PPO on humanoids when it is scaled the same way: thousands of worlds, huge batches, a GPU replay buffer. They trained sim-to-real humanoid walking in 15 minutes on one RTX 4090. It had not been tried on a quadruped like this one.

Same worlds, different learner. `train_sac.py` uses the same tasks, rewards, scenario banks, held-out tests and output files as `train.py`; only the learning rule changes. The exported actor runs in the same `policy.py`.

15.1 Idea 1: learn Q and keep a replay buffer

Every step of every world is stored as $(o, p, a, r, o', p', \text{end})$ (p: privileged state) in a ring buffer on the GPU: 1,048,576 transitions, i.e. the last 256 steps (5 s) of each of 4,096 worlds. Each world step adds 4,096 transitions and the learner then makes 4 updates of 16,384 random samples: 65,536 samples, so every transition is learned from about 16 times before it falls out. (PPO: 5 times.) The critic learns $Q(s, a)$ by the Bellman target (§4), so it can learn from actions an older policy took: the target asks what the *current* policy would do next, $a' \sim \pi(o')$.

15.2 Idea 2: the actor follows the critic's slope

PPO only needs $\log \pi$ of the action taken. Here the critic is a differentiable function of the action, so the actor is improved by gradient ascent *through* it (DDPG [9]). With the *reparameterisation* trick the sampled action is a function of the network and fixed noise ϵ:

$$u = \mu_\theta(o) + \sigma_\theta(o)\,\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \qquad a = \tanh(u) \in [-1, 1]^{12},$$

so $\partial Q(s, a)/\partial\theta$ flows from the critic, through a, into μ and σ. Unlike PPO's actor, this one outputs its own noise level per state (24 outputs: 12 means, 12 log-stds).

15.3 Idea 3: maximum entropy (the "soft" in SAC)

SAC [13] pays the policy for being random as well as for reward, with a temperature α:

$$J(\pi) = \mathbb{E}\Big[\sum_t \gamma^t \big(r_t + \alpha\,\mathcal{H}[\pi(\cdot \mid o_t)]\big)\Big], \qquad \mathcal{H} = -\mathbb{E}_{a\sim\pi}[\log \pi(a \mid o)].$$

That changes both losses (`rl2/train_sac.py` L174-L206):

$$y = r + \gamma\Big(\tfrac{1}{2}\textstyle\sum_{i=1,2} \bar{Q}_i(s', a') - \alpha \log \pi(a' \mid o')\Big), \qquad L_\pi = \mathbb{E}\big[\alpha \log \pi(a \mid o) - \tfrac{1}{2}\textstyle\sum_i Q_i(s, a)\big],$$

where $\bar{Q}$ is the target network. The actor wants high Q *and* high entropy; α sets the exchange rate. Instead of guessing α, it is tuned automatically [14] so the entropy sits at a target $\bar{\mathcal{H}}$: $L(\alpha) = -\alpha\,(\bar{\mathcal{H}} - \mathcal{H})$. Entropy below target ⇒ α rises ⇒ more exploration.

```
g = jax.grad(lambda la_: -la_ * (o["ent_target"] - ent))(params["log_alpha"])   # entropy < target -> alpha up
```

The tanh correction. Squashing changes the density: $\log \pi(a) = \log \mathcal{N}(u; \mu, \sigma) - \sum_j \log(1 - \tanh^2 u_j)$. The code uses the stable form $\log(1 - \tanh^2 u) = 2(\log 2 - u - \text{softplus}(-2u))$ (`rl2/train_sac.py` L90-L97). Forget it and the entropy, and so α, are wrong.

15.4 Idea 4: two critics, and how to combine them

Q-learning *overestimates*: the actor seeks actions where the critic is wrongly high, and the error is copied into every target built from it [8]. TD3 [12] trains two critics and builds targets from the *lower* one ("clipped double Q"). FastSAC found that with layer norm in the critics (which already curbs blow-ups) the minimum is too pessimistic and hurts: it under-rates every unfamiliar action, so the actor stops trying them. It uses the *average* instead [20]; related work on scaling regularised critics argues the same way [18]. The code keeps both (`--cdq`).

15.5 Idea 5: a distribution, not a number (C51)

Each critic outputs probabilities over 101 fixed return values (*atoms*) from -10 to 50, spacing $\Delta z = 0.6$ [10]. Q is their mean. The Bellman target moves each atom to $r + \gamma z$ and splits it between the two nearest atoms (`rl2/train_sac.py` L142-L150); the critic is trained by cross-entropy against that. Example: atom $z = 4.4$, $r = 0.3$, $\gamma = 0.97$: $0.3 + 0.97 \times 4.4 = 4.568$, which is 0.28 of the way from atom 4.4 to atom 5.0, so 72 % of its probability goes to 4.4 and 28 % to 5.0 ($0.72 \times 4.4 + 0.28 \times 5.0 = 4.568$, the mean is preserved). The entropy term shifts every atom by $-\alpha \log \pi$ first. Why bother: learning the spread of outcomes (a fall or not) gives the critic richer gradients than one averaged number, and cross-entropy is better behaved than squared error on a moving target.

15.6 Idea 6: n-step returns

A 1-step target trusts the critic after a single real reward, so news of a fall travels back one step per update. Adding n real rewards first (`rl2/train_sac.py` L152-L172):

$$y = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 \bar{Q}(s_{t+3}, a'),$$

stopping early if the episode ends (and not bootstrapping at all after a real ending; a time-out still bootstraps). Strictly this is biased off-policy (the 3 rewards came from an older policy), but with a 5 s buffer the policy that took them is recent. Rainbow [11] combined it with the distributional critic; FastTD3 uses 3, and so do we.

15.7 Idea 7: scale, and the settings that made it work

Setting	Value, and why
Worlds, batch, updates	4,096 worlds; 4 updates of 16,384 per world step; replay buffer 1 M (all on the GPU, one jitted call per 25 steps)
Critics	$2 \times (203 + 12 \to 768 \to 384 \to 192 \to 101)$ with layer norm [16]; asymmetric: read the privileged state [15]
Actor	$153 \to 512 \to 256 \to 128 \to 24$, sensors only; deterministic $\tanh \mu$ on the robot
Target network	Polyak average $\bar{\phi} \leftarrow 0.9\,\bar{\phi} + 0.1\,\phi$ every update
Target	**average of the two critics** (not the minimum)
Entropy	**target 0, initial** $\alpha = 0.001$**, pre-tanh std** ≤ 1: too much noise early wrecked the first runs
Discount	$\gamma = 0.97$ (horizon $\approx$0.7 s)
Optimiser	**AdamW, lr** 3×10^{-4}**, weight decay 0.001,** $\beta_2 = 0.95$ [17]; gradient-norm clip 1

The FastSAC loop, one call of `chunk()` (`rl2/train_sac.py` L214-L245), repeated 25 times inside the GPU:

1. every world samples $a = \tanh(\mu + \sigma\epsilon)$ from the actor and steps the physics (as PPO);
2. the 4,096 transitions go into the buffer, the input normaliser is updated, finished worlds respawn;
3. 4 times: draw 16,384 3-step samples; build the C51 target from the averaged target critics; one critic step, one actor step through the new critics, one α step, Polyak-update the target critics.

No rollout-then-update rhythm: acting and learning interleave every step. Learning starts after 0.5 M transitions.

15.8 What happened (Experiment 6, queue6–7, 26 Oct 2026)

Terrain, 3 seeds each, held-out	Peak, GPU (4,160)	Best snap., CPU (200)	Last snap., CPU (200)
PPO, 400 M steps, ~30 min	63–71 %	62.5–77.0 % (mean 70.7)	55.5–60.0 %
FastSAC, my first settings (3-step)	32–52 %	34.0–55.0 % (mean 45.2)	–
FastSAC, paper settings, 150 M, ~34 min	**83–90 %**	**83.0–91.0 % (mean 88.2)**	**83.0–90.5 %**

Stress audit of the best snapshots (success per terrain type, 3 seeds, three levels): PPO / FastSAC: flat .92/1.0, rough .86/1.0, slope .80/.94, stairs up .78/.96, stairs down .64/.98, gaps .05/.41, obstacles .75/.98, mixed .59/.86; overall **.67 / .89**.

- **The settings were the difference.** The same code lost to PPO by 25 points with textbook SAC settings (min of two critics, entropy target $-|A|/2$, $\alpha_0 = 0.01$, std up to e) and won by 17 with the paper's. The early runs swung ±15 points between evaluations; the new ones climb smoothly. Which single setting mattered most is not yet separated; that ablation is a next step.
- **No late slump**: last snapshot within 2.5 points of the best on all seeds. The slump was PPO's, not the task's.
- **Fast**: seed 0 passed PPO's best peak after 16 M steps, about 4 minutes.
- **Blind, it beats the teacher–student pair** (77.0 %) and the sighted teacher's peak (80.9 %), with a plain 3-frame MLP. Gaps remain hard (.05 at the hardest level): no amount of learning sees a gap without a sensor.
- **Cost**: 74 k steps/s against PPO's 228 k: the GPU spends most of its time learning, not simulating. Per minute of wall-clock it still wins, because it learns far more from each step.

	PPO (`train.py`)	**FastSAC** (`train_sac.py`)
Critic learns	$V(s)$ from real returns (GAE)	two $Q(s,a)$ distributions from 3-step Bellman targets
Actor improves by	raising $\log \pi$ of better-than-expected actions, clipped	gradient of Q through the action, plus α·entropy
Exploration	one learned std per joint; entropy bonus c_e	per-state std; α tuned to hold an entropy target
Data	196,608 fresh samples per update, used 5 times	1 M-sample buffer, each sample used ~16 times
Step size control	clip ±20%, adaptive lr from KL	fixed lr, slow target network, layer norm
Here, on terrain	70.7 %, slumps late	88.2 %, stable

Where this goes next. (1) FastSAC for the other skills (walk, handstand, rear, balance), running now; (2) the ablation: one paper setting at a time; (3) a sighted FastSAC teacher distilled into a student with a forward range sensor, for gaps; (4) *RL Token* [28]: freeze a good policy, read its last hidden layer as a compact "token" of the situation, and train a small residual actor-critic on top that only nudges its actions, with a penalty for straying. That is how large pretrained manipulation policies are being fine-tuned with RL on real robots; here it could fine-tune on the real quadruped without retraining the whole network.

16 Is it good? How a policy is judged

- **Held-out, noise off.** Training success is measured with exploration noise on scenarios being trained on. The number that counts is success on scenarios never trained on, with the mean action.
- **The full model.** Training runs in the light GPU model; every reported score is re-measured in the full CPU model (every mesh, rotors on their joints), which the policy never saw.
- **Pick the snapshot, not the last iterate.** Held-out success swings ±15 points between evaluations 40 M steps apart and often sags late. `pick_best.py` plays the top GPU snapshots on a separate CPU *selection* bank, keeps the winner, and only then `play.py eval` gives the test number. This was the biggest single gain: terrain 50.5 → 73.5 % (3 seeds), handstand 10 → 82 %.
- **Stress audit** (`audit.py`): worst-case sensors, weak motors, slippery floor, dead flow sensor.
- **Seeds.** One run is one sample; differences under ~5 points between single runs are noise.

17 A new build version

`rl2` never names v4: it reads the robot from the folder it sits in (`core.BUILD`, or `CAPSTAN_BUILD=<path>`).

1. Build v5 as usual (CAD → meshes → `urdf/build_urdf.py`), keeping the names: `torso`, `foot_LF`…`foot_RR`, rotor joints `*_rotor*`.
2. Copy the code: `rsync -a --exclude policies --exclude logs fullbuild_v4/rl2/ fullbuild_v5/rl2/`.
3. Check it: `JAX_PLATFORMS=cpu python check_robot.py --policy=walk2_best`. Names, both models, standing on PD alone, sensors in range, and whether an old policy still walks. If that fails, retrain.
4. If the robot changed a lot, revisit the v4-tuned numbers: PD gains and action range in `core.py`, sensor values in `hw.py`, fall heights and pose targets in `tasks.py`.
5. Train on cake: `sh run_queue.sh experiments/<batch>.list`. Films come out as `video/v5_rl2_*.mp4`.

18 Words

Policy / actor	The network that picks actions; the only part that runs on the robot.
Critic / value	Predicts future reward. Training only, so it may read privileged state.
Return	Sum of future rewards, each step ahead counted 0.99× less.
Advantage	Return actually got minus what the critic expected. Positive: do that more.
GAE	Advantage estimate mixing real rewards with the critic's guess; λ trades bias for variance.
Entropy, `log_std`	How much noise the policy adds to explore. Learned; a bonus keeps it from collapsing early.
KL	How far one update moved the policy; used to adapt the learning rate.
Sparse / dense reward	Paid only at the end (success, fall) vs every step (tracking, posture).
Shaping / tuning	Choosing reward terms / choosing their weights.
Domain randomisation	Training on many versions of the world so the policy works on the real one.
Scenario bank	Fixed list of episode settings; training draws from one, testing uses another.
Privileged information	What the simulator knows and the robot cannot sense (terrain ahead, friction).
Teacher–student, DAgger	Train with privileged info, then imitate it from real sensors, on the student's own states.
Held-out evaluation	Testing on scenarios never trained on, without noise. The only fair score.
Sim-to-real gap	Everything the simulation gets wrong about the robot; `hw.py` exists to shrink it.
Q-function	Expected return after taking action a in state s. FastSAC's critics learn it.
Bellman target, TD	$r + \gamma Q(s', a')$: train a value toward one real reward plus its own next prediction.
Target network	A slowly averaged copy of the critic used for targets, so they do not chase themselves.
On-/off-policy	Learns only from the current policy's fresh data (PPO) / from any stored data (FastSAC).
Replay buffer	Store of past transitions that off-policy methods sample from, again and again.
Temperature α	SAC's price of entropy; tuned automatically to hold an entropy target.
Reparameterisation	Writing a sampled action as $\tanh(\mu + \sigma\epsilon)$ so gradients pass through it.
Distributional (C51)	A critic that predicts probabilities over 101 return values, not a single mean.
n-step return	n real rewards before trusting the critic; spreads news of a fall back faster.

Results, films and the whole run history: `rl2/sgs-lessons.md`; the results pages on claude.ai; the `capstan-rl2` project on Weights & Biases.

19 Check yourself

Cover the answers. If one is hard, the section in brackets has it.

1. *Why can a robot that stands still collect plenty of reward and still learn nothing from it?* Learning uses the advantage, reward relative to what the critic expected; standing gets exactly what it expects, so $A \approx 0$. (§5)
2. *What is the difference between $V(s)$ and $Q(s, a)$, and which one does each of our trainers learn?* V scores the situation, Q the situation plus a chosen action. PPO learns V, FastSAC learns Q. (§4)
3. *Write the 1-step Bellman target for Q. Why is a separate target network needed?* $y = r + \gamma Q(s', a')$; without a slowly moving copy, the target shifts with every update and errors amplify. (§4)
4. *PPO throws its data away after 5 passes. Why can't it keep it like FastSAC?* Its gradient and GAE assume the data came from the current policy; old data from a different policy gives a wrong gradient. FastSAC's Bellman target asks the current policy for a', so old transitions stay valid. (§4, §15)
5. *How does PPO's actor get better, and how does FastSAC's?* PPO: raise the log-probability of actions with positive advantage (clipped). FastSAC: move the action up the critic's slope $\partial Q/\partial a$, via the reparameterised sample, plus an entropy bonus.
6. *The logged entropy is below its target. What happens to α, and then to the policy?* α rises, the entropy term weighs more in the actor loss, the policy widens its noise until the entropy is back at target.
7. *Why use the average of two critics here rather than the minimum (TD3)?* With layer-normed critics overestimation is already small, so the minimum under-rates unfamiliar actions and the actor stops exploring. Measured here: min 45 %, average (with the other paper settings) 88 %.
8. *The atom at $z = 10.4$ gets $r = 0.2$, $\gamma = 0.97$. Where does its probability go (atoms every 0.6 from -10)?* $0.2 + 0.97 \times 10.4 = 10.288$, which is $0.488/0.6 = 0.81$ of the way from atom 9.8 to atom 10.4: 19 % to 9.8, 81 % to 10.4. (§15)
9. *Why does a time-out bootstrap but a fall does not?* A fall really ends the future (value 0). A time-out is an artificial cut; the robot would have kept collecting reward. (§5)
10. *Why can't the teacher run on the robot, and why is DAgger better than copying the teacher's trajectories?* It reads a height scan and physics the robot cannot sense. DAgger trains on states the student itself visits, including after its own mistakes. (§14)
11. *A run's held-out success went 71 % → 55 %. Which file do you deploy?* `<name>_best.npz` from `pick_best.py`, chosen on a separate selection bank, then tested on the held-out bank.

References

[1] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. http://incompleteideas.net/book/the-book-2nd.html

[2] J. Achiam. *Spinning Up in Deep RL*. OpenAI, 2018. https://spinningup.openai.com

[3] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning (REINFORCE). *Machine Learning*, 1992.

[4] J. Schulman et al. Trust Region Policy Optimization. ICML 2015. arXiv:1502.05477

[5] J. Schulman et al. High-Dimensional Continuous Control Using Generalized Advantage Estimation. ICLR 2016. arXiv:1506.02438

[6] J. Schulman et al. Proximal Policy Optimization Algorithms. 2017. arXiv:1707.06347

[7] V. Mnih et al. Human-level control through deep reinforcement learning (DQN). *Nature*, 2015.

[8] H. van Hasselt. Double Q-learning. NeurIPS 2010.

[9] T. Lillicrap et al. Continuous control with deep reinforcement learning (DDPG). ICLR 2016. arXiv:1509.02971

[10] M. G. Bellemare, W. Dabney, R. Munos. A Distributional Perspective on Reinforcement Learning (C51). ICML 2017. arXiv:1707.06887

[11] M. Hessel et al. Rainbow: Combining Improvements in Deep Reinforcement Learning (n-step + distributional). AAAI 2018. arXiv:1710.02298

[12] S. Fujimoto, H. van Hoof, D. Meger. Addressing Function Approximation Error in Actor-Critic Methods (TD3). ICML 2018. arXiv:1802.09477

[13] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep RL with a Stochastic Actor. ICML 2018. arXiv:1801.01290

[14] T. Haarnoja et al. Soft Actor-Critic Algorithms and Applications (automatic temperature). 2018. arXiv:1812.05905

[15] L. Pinto et al. Asymmetric Actor Critic for Image-Based Robot Learning. RSS 2018. arXiv:1710.06542

[16] J. L. Ba, J. R. Kiros, G. E. Hinton. Layer Normalization. 2016. arXiv:1607.06450

[17] I. Loshchilov, F. Hutter. Decoupled Weight Decay Regularization (AdamW). ICLR 2019. arXiv:1711.05101

[18] M. Nauman et al. Bigger, Regularized, Optimistic: scaling for compute and sample-efficient continuous control. NeurIPS 2024. arXiv:2405.16158

[19] Y. Seo, C. Sferrazza, H. Geng, M. Nauman, Z.-H. Yin, P. Abbeel. FastTD3: Simple, Fast, and Capable Reinforcement Learning for Humanoid Control. 2025. arXiv:2505.22642

[20] Y. Seo, C. Sferrazza, J. Chen, G. Shi, R. Duan, P. Abbeel. Learning Sim-to-Real Humanoid Locomotion in 15 Minutes (FastSAC). 2025. arXiv:2512.01996

[21] N. Rudin, D. Hoeller, P. Reist, M. Hutter. Learning to Walk in Minutes Using Massively Parallel Deep RL. CoRL 2021. arXiv:2109.11978

[22] J. Lee et al. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 2020. arXiv:2010.11251

[23] A. Kumar, Z. Fu, D. Pathak, J. Malik. RMA: Rapid Motor Adaptation for Legged Robots. RSS 2021. arXiv:2107.04034

[24] S. Ross, G. Gordon, D. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (DAgger). AISTATS 2011.

[25] G. Margolis, P. Agrawal. Walk These Ways: Tuning Robot Control for Generalization with Multiplicity of Behavior. CoRL 2022. arXiv:2212.03238

[26] Z. Fu, A. Kumar, J. Malik, D. Pathak. Minimizing Energy Consumption Leads to the Emergence of Gaits in Legged Robots. CoRL 2021. arXiv:2111.01674

[27] Zhang et al. A Balanced Data Diet (SGS). 2026. arXiv:2610.12465

[28] C. Xu et al. RL Token: Bootstrapping Online RL with Vision-Language-Action Models. 2026. arXiv:2604.23073